

Chapter 9. Data Transformation and Delivery Layer

By now, you should be familiar with the first two layers of the financial data engineering lifecycle: ingestion and storage. The next layer is the transformation and delivery layer, where two major things happen: first, the data undergoes a range of transformations, from basic preprocessing and cleaning to complex analytical calculations; second, the data is delivered to its end users following pre-established agreements. Keep in mind that all layers can impact one another based on business needs and technical limitations. As a result, the decisions you make in the ingestion and storage layers are likely to have an impact on this layer and vice versa.

Throughout this chapter, you will learn the essential concepts of data transformation and delivery and their applications in the financial domain. This includes data querying patterns, query optimization, transformations, computational requirements, data consumers, and delivery mechanisms.

Data Querying

The most common operation performed in the transformation and delivery layer is data querying. Before making any modifications, data is always queried. The performance of a financial data infrastructure is largely dependent on querying patterns and optimization. Therefore, as a financial data engineer, you will play a vital role in defining and optimizing the querying needs and patterns for your team and data consumers. Let's find out how.

Highlight

Add Note

Copy

Querying Patterns

A data querying pattern is a repeated request for information made by several data consumers on a regular basis. A pattern can be either detected by analyzing user requests or anticipated during an early design phase. Being able to understand and anticipate querying patterns will provide you with great input for the choice and design of your data storage model as well as optimizing the cost and execution time of your data queries.

There is no fixed catalog for querying patterns; they are primarily determined by business requirements and data consumer needs. However, in this section, I will introduce several common querying patterns that you might encounter in financial applications.

Time series queries

Time series queries are essential in finance, used to retrieve data for a specific financial entity or quantity over a given period of time. Using standard SQL pseudocode, we can express a basic time series query:

```
-- SQL
SELECT time_column, attribute_1, attribute_2
FROM financial_entity_table
WHERE entity_name = 'A'
AND date BETWEEN '2022-01-01' AND '2022-02-01'
```

Financial use cases for time series queries include the following:

- Give me the closing adjusted price for stock A from January 2022 until February 2022.
- Give me all transactions for client A for the past month.
- Give me a company's sales and revenues over the past six months.

Cross-section queries

A cross-section query is used to obtain data for a set of financial entities at a specific point in time. A simplified SQL pseudocode for a cross-section query might look something like this:

```
-- SQL
SELECT entity_name, attribute_1, attribute_2
FROM financial_entity_table
WHERE entity_name in ('A', 'B', 'C', 'D')
AND date = '2022-02-01'
```

Cross-section queries are commonly used to investigate differences between financial entities at a given point in time. Examples include the following:

- Give me the market capitalization of the top five companies listed at the NYSE for February 1, 2022.
- Give me the credit rating of all publicly traded companies in the pharmaceutical sector in the United States on January 10, 2022.

Panel queries

A panel query combines time series and cross-section dimensions. The user asks for data on multiple financial entities for a range of dates. A pseudo-SQL for a panel query may look like this:

```
-- SQL
SELECT time_column, entity_name, attribute_1, attribute_2
FROM financial_entity_table
WHERE entity_name in ('A', 'B', 'C', 'D')
AND date BETWEEN '2022-01-01' AND '2022-02-01'
```

Panel queries are mainly used to analyze the intertemporal differences between financial entities—for example the following:

- Give me the online purchasing activities of our top 1,000 clients in the past month.
- Give me the volume of trades for stocks A, B, and C over the past month.

Analytical queries

Analytical queries are advanced statements that perform computations on the data. They are often supported and used in SQL and data warehousing systems as well as specialized databases such as time series and graph databases.

The most common type of analytical query is grouping, in which rows that have the same values are grouped into summary rows using an aggregation function such as sum, min, max, average, standard deviation, and others. For instance, if you want to find the maximum daily price for each unique combination of stock symbols and dates in your database, you would write an SQL query similar to this:

```
-- SQL
SELECT stock_symbol, date, MAX(price)
FROM price_table
GROUP BY stock_symbol, date
```

Grouping reduces the number of rows into a set of groups based on your aggregation fields. In some cases, you don't want the number of rows to shrink, but rather want to apply an aggregation query that spans multiple time windows. For example, suppose you want to calculate the simple moving average of the adjusted closing price for each stock in your dataset.¹ In this case, a special type of SQL method, called *window functions*, can be used. Here is how it works:²

```
-- SQL
create table adjsted_price (date DATE, symbol VARCHAR(5), price NUMERIC);
insert into adjsted_price (date, symbol, price) values ('2022-02-10', 'A', 1);
insert into adjsted_price (date, symbol, price) values ('2022-02-11', 'A', 2);
insert into adjsted_price (date, symbol, price) values ('2022-02-12', 'A', 3);
insert into adjsted_price (date, symbol, price) values ('2022-02-13', 'A', 4);
insert into adjsted_price (date, symbol, price) values ('2022-02-10', 'B', 10);
insert into adjsted_price (date, symbol, price) values ('2022-02-11', 'B', 20);
insert into adjsted_price (date, symbol, price) values ('2022-02-12', 'B', 30);
insert into adjsted_price (date, symbol, price) values ('2022-02-13', 'B', 40);

SELECT symbol, date, AVG(price)
OVER (PARTITION BY symbol ORDER BY date ASC ROWS BETWEEN 2
PRECEDING AND CURRENT ROW) AS "Moving Average"
FROM adjsted_price
ORDER BY symbol, date ASC
```

A variety of data storage systems implement window and analytical functions. For example, relational DSSs like PostgreSQL implements the SQL standard window functions, while others such as Oracle implement [addition functions](#) on top of the standard ones. Specialized databases, such as InfluxDB time series database, offer an impressive [collection of time-based analytical functions](#). Similarly, graph databases such as Neo4j offer a wide range of [graph-oriented functions](#) that can be executed with the Cypher language, as well as a specialized library called Graph Data Science that offers advanced graph algorithms and visualization features that can be used to solve advanced business problems such as fraud detection.

Recently, cloud data warehouse solutions such BigQuery and [Snowflake](#) have introduced features enabling users to create and interact with machine learning models using advanced analytical SQL queries. These capabilities allow users to build various models (e.g., regression, classification, anomaly detection, etc.) and make predictions on new data, all seamlessly integrated within the data warehouse environment. Using [BigQuery](#), the syntax for creating a model may look like the following:

```
-- SQL
-- Create a linear regression model
CREATE MODEL `mydataset.my_model`
OPTIONS(MODEL_TYPE='LINEAR_REG') AS
SELECT
  feature_column1,
  feature_column2,
  label
FROM
  `mydataset.my_training_table_data`;
```

When calling the model, the syntax would look like this:

```
-- SQL
SELECT *
FROM
  ML.PREDICT(MODEL `mydataset.my_model`, (
    SELECT
      feature_column1,
      feature_column2
    FROM
      `mydataset.my_test_table_data`
  ));
```

Before moving to the next section, an essential thing to keep in mind is that data querying needs can evolve and change over time. It's impractical and costly to switch data storage systems due to query limitations. Therefore, it's essential to consider the broader context, prioritize the needs of the business and data consumers, and develop a strategy to manage both current and future query requirements effectively.

Query Optimization

Once you've identified your querying needs and patterns, the next step is to think about and implement a query optimization strategy. If queries are not optimized, their performance can be significantly impacted. For simplicity, I will define performance in terms of query speed and cost. A query optimization strategy can be approached from two perspectives: the database side and the user side. Let's explore each in detail.

Database-side query optimization

On the database side, there are three main query optimization techniques: indexing, partitioning, and clustering. Indexing is the most common technique. To understand the need for indexing, let's illustrate its main use case. Imagine you have a database that contains price data for 1,000 stocks. You want to run a time series query that fetches data for one stock (stock A). The database engine doesn't know where the data for stock A is physically located on disk. For this reason, the only way to execute the query is to read all data for all stocks and keep stock A's records only. This type of data read is called *full-scan*, as the entire table is scanned. As you can imagine, this is an expensive operation, given the time it takes to do a full scan. Can we do better? Yes, we need to add an index!

An index is an optimized data structure that holds information about the disk location of the data stored in your database. When the right index is added, the database engine will try to use it to look up the location of the data and avoid querying unnecessary records. Now you might wonder, how do I know which index to add? The answer depends on your querying patterns. A rule of thumb is to add an index on the columns that you use in your `WHERE` clause. For example, let's consider the following query:

```
-- SQL
SELECT time_column, attribute_1, attribute_2
FROM financial_entity_table
WHERE entity_name = 'A'
AND date BETWEEN '2022-01-01' AND '2022-02-01'
```

As you can see, the `WHERE` clause uses two columns: `entity_name` and `date`. For this reason, it makes sense to add an index on both columns. In this case, the index is called *composite* as it is created on multiple columns. If your query were to only filter by `entity_name`, then it's better to create a single-column index instead. Now you might ask, how do I know what type of index I need to add? There are indeed a variety of database indexes. To illustrate with an example, let's take PostgreSQL as a reference as it has a [large variety of index types](#).

The standard index implementation in PostgreSQL is *B-tree*, a tree-based data structure that achieves logarithmic complexity.³ The reason behind the popularity of B-tree indexes is their ability to handle a wide range of query operators: less than (`<`), less than or equal to (`<=`), equal to (`=`),

greater than (>), greater than or equal to (>=), and between. Other types of data storage systems, such as MongoDB and Neo4j, use B-tree as their default index implementation. Therefore, unless you have any special query needs, the B-tree is generally a good indexing strategy. Keep in mind, however, that indexes are stored on disk and grow in size as your data does. In addition, indexes get updated as you update, insert, and delete records from the database. For this reason, you must aim to create the least number of indexes when possible.

Other types of indexes are often used for more specific use cases. For example, the *Block Range Index* (BRIN) is a lightweight index that can be used with very large tables, where the indexed columns have a strong correlation with the physical order of data in the table. A typical scenario might involve a large stock price time series dataset where data is organized by date. When performing time series queries involving a wide temporal range (e.g., last month), PostgreSQL can utilize the BRIN index to quickly identify and skip over data blocks that do not contain relevant dates.

In some cases, indexes such as B-tree and BRIN are complemented with clustering, which involves physically rearranging data based on one or more columns, often those used to create an index. This rearrangement is advantageous when your queries frequently access a range of ordered values, such as in time series queries. It's important to remember that clustering is a one-time operation. If the data is updated, re-clustering will be required.

Last but not least, a powerful database optimization technique is *partitioning*. It works by dividing the data into logical and physical partitions/tables using one or more partition keys. Queries filtering by these keys will exclusively scan relevant partitions, minimizing unnecessary data access. A partition key is often chosen to minimize the number of scanned partitions. For example, suppose you have high-frequency stock price data that arrives daily, and assume that most of your queries include a day range filter (e.g., the last three days, first 10 days of last month,...). In this case, it makes sense to partition your table by date to have one partition for each day. This way, even if you have 1,000 partitions, if you query data for one date, then only one partition will be scanned.

You might ask yourself, how do databases figure out the most optimal way to execute a query and choose which index and scanning strategy to perform? This is all done by the so-called query planner/optimizer.

Each database system implements its own query planner, which is responsible for figuring out the most efficient execution plan for each user query. Typically, query planners represent queries as trees, allowing for multiple execution strategies that yield identical results. These trees are typically read from the bottom up, with data being retrieved initially from one or more locations, and then aggregated as you move upwards to produce the final results. The query planner's goal is to select the optimal plan for execution, which may involve substituting the original query with one or more optimized versions to enhance performance. Some databases implement more complex techniques. For example, PostgreSQL's query planner might resort to an advanced optimization technique based on genetic algorithms when planning for complex queries (e.g., when there is a large number of join operations).

Common strategies that query planners often rely on include the following:

Sequential scan

Scans the entire table. This is used when the table is small or if indexing is not properly done.

Index scan

Performs an index traversal to find all matching records. The planner might perform an index scan if any index satisfies the `WHERE` condition.

Index-only scan

Performed if all queried columns are part of an index, returning tuples directly from index entries.

Parallel scan

Multiple processes fetch a subset of the requested data simultaneously, thereby speeding up query execution.

Partition pruning

Used with partitioned database systems (e.g., BigQuery and Snowflake) to minimize the number of partitions to scan.

Block pruning

Used mostly with clustered tables, and determines which blocks of data to read, thus saving disk access and accelerating query execution.

Many database systems offer a command called `EXPLAIN`, which provides a detailed breakdown of the execution plan that the query planner

generates for a given query statement. To read more about query planners, I recommend having a look at PostgreSQL's [Planner/Optimizer](#), BigQuery's [Dremel engine](#), and Redshift's [query optimizer](#).

In conclusion, it is essential to remember that database optimization may be a time-consuming and challenging task (yet fun as well!). As such, give it some serious thought and include the business team to ensure that all of their demands are met. Furthermore, be sure to provide a summary of all the technical limitations that may arise along the way, and consider how they can affect future data and querying needs.

User-side query optimization

Database-side optimization is only half the story; the other half concerns optimizing the way users interact with the data. You can have the best indexing, clustering, and partitioning strategy in place, but users need to follow the right querying approach to benefit from such optimizations. There is no unique recipe for telling users how to query the data—it's all based on the querying needs and the database optimization put in place. Let me give you a few illustrative examples.

Scenario 1

If you add a single-column index on column A, try to avoid queries that do not reference column A in their search conditions. While there's no guarantee the query planner will use the index every time, doing so could potentially result in notable performance gains.

Scenario 2

If you add a composite index on columns A, B, and C, then make sure that you filter by the leading (leftmost) columns such as [A], [A, B], or [A, B, C]. Filtering by [B] or [C] or [B, C] will lead to inefficient index scans. I highly recommend that you read more about this topic, and the PostgreSQL [documentation page](#) is a good place to start.

Scenario 3

When querying, select only the necessary columns rather than performing `SELECT *`. This is particularly relevant when working with column-oriented databases that store data on disk column-wise. In this case, querying only a small subset of the columns will significantly improve your query speed and reduce inefficient data reads.

Scenario 4

If you use the SQL pattern matching operator `LIKE`, then try to anchor the constant string at the beginning; for example, do `column LIKE 'ABC%'`, but not `column LIKE '%ABC'`. With `ABC%`, an index such as B-tree is very likely to know which records to consider as it knows what they start with, but with `%ABC`, the index doesn't know which strings end

with ABC, and it might need to scan the full index. You can read more about this topic in the [PostgreSQL documentation](#).

Scenario 5

When processing large amounts of data, try to modularize your queries. For example, say you have a very large stock price time series table and you want to perform a given transformation on the entire table. In this case, consider using an incremental loading and processing approach instead of applying your operation to the entire table in one run. For instance, you can query your data one day or one month at a time and apply the transformations on each batch separately. The main advantage of this approach is that in case of a failure with one batch, you don't need to query the entire dataset again.

Scenario 6

When performing complex queries that involve joins and aggregations, make sure to process a minimal amount of data. For instance, consider a scenario where you need to get monthly transaction amounts for specific customers using two tables: transactions and customers. As illustrated in [Figure 9-1](#), one way to do this is by first joining the transaction and customer tables, then aggregating the results by month and customer, and finally filtering for the desired customers. One issue with this approach is that you might join potentially large tables, which can be a resource- and time-intensive operation. A more efficient strategy would involve delaying the join operation to a later stage and applying filtering up front to reduce the queried dataset size.

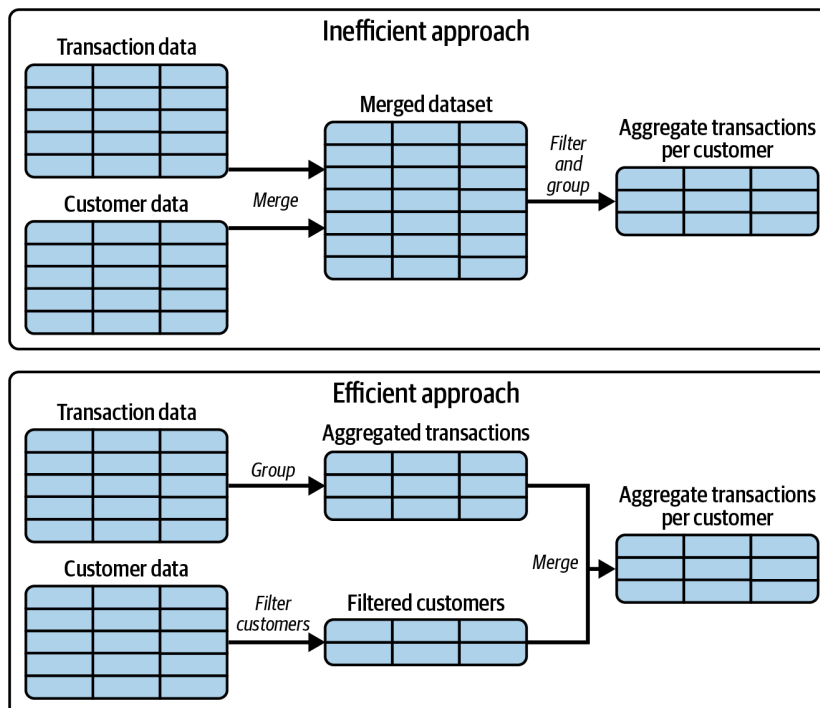


Figure 9-1. An inefficient versus efficient data processing approach

Based on your institution's requirements and the needs of data consumers, you, as a financial data engineer, should be able to anticipate and

assess additional scenarios and use cases. I recommend consistently striving to understand and discover ways to improve the efficiency of your queries. Use the `EXPLAIN` command to analyze the query optimizer plans, try to learn about database internals and optimization strategies, and pick up some advanced SQL knowledge.⁴

Data Transformation

Once you have determined your querying strategy and optimized your database accordingly, the next step is to develop and implement your data transformations. The main purpose of data transformations is to prepare data for various use cases across your organization's departments and teams. In this section, you will learn about the different types of transformation operations and patterns commonly used in finance, along with the computational requirements essential for their implementation.

Transformation Operations

In its most basic form, a data transformation involves converting a raw, unprocessed dataset into a structured format suitable for its intended business application. Importantly, there isn't a universal list of transformations that you need to apply to your data. Therefore, as a financial data engineer, one of your key responsibilities will be to discuss and define the specific transformations to implement based on business requirements and the needs of data consumers.

WARNING

Avoid applying any transformation directly to the raw data. Instead, store the raw data in an archive location and apply your transformations on a copy of the data.

I'll provide a general overview in the following sections, outlining some of the fundamental transformations typically applied to financial data.

Format conversion

The first transformation applied to financial data typically involves format conversion. It consists of converting the data from its source format into another format that is easier to work with. For example, you might convert data in CSV format to SQL tables, or convert data in JSON format into a collection in a document database. Once the data is converted into the desired format, subsequent transformations become easier. [Figure 9-2](#) illustrates a common format conversion scenario in the financial industry, where raw data arriving in CSV or XLSX (Excel) formats is transformed into a tabular format within a relational database.

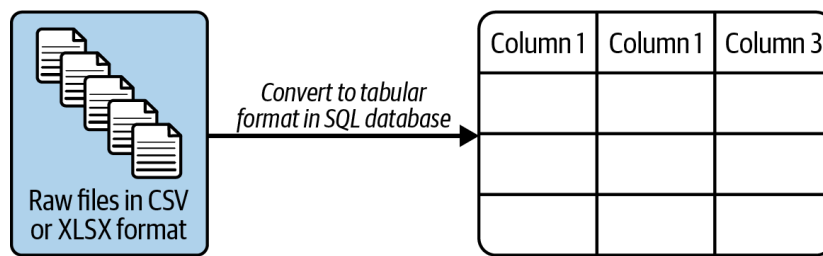


Figure 9-2. An example of format conversion

Data cleaning

Data cleaning involves the detection and correction of data quality issues such as errors, biases, duplicates, invalid formats, outliers, incorrect/corrupt values, missing data, and many others.

Data cleaning is a crucial step in the transformation layer, and it should be handled carefully. First of all, you need to make sure that all data quality issues are properly identified and understood. This assessment can vary based on your business problem; therefore, it needs to be discussed with the business team and the data consumers.⁵ Once an agreement has been reached, quality checks and corrective measures need to be put in place. The nature of these remedial measures is often dependent on the severity of the data quality issue. For example, in the context of fraud detection, a data outlier might signal potentially suspicious activity that should be investigated further. But if we take intraday high-frequency price data, you could easily find outliers, but they often don't have a significant impact.

Determining when and which data to clean within your financial data infrastructure is a critical aspect of data integrity. While it may seem intuitive to perform cleaning as early as possible, this approach requires careful consideration. Often, the rationale for cleaning early is that downstream systems may struggle to identify data issues originating from upstream sources, especially if they lack the necessary contextual information to diagnose the problem.

Consider high-frequency market data as an example. While academic research often highlights the importance of data cleaning and anomaly detection for such data, advances in modern data infrastructure are reducing this need. Electronic trading has greatly reduced data anomalies, and data feed providers receive quick feedback from clients—particularly trading firms that promptly test new feeds, protocols, and connections. This immediate feedback allows for quick fixes, meaning that data feeds may not always need cleaning.

However, there are exceptions. For example, if you're working with data from a redistributor that modifies the raw data, cleaning becomes essential. This is also the case for OTC markets or those with outdated systems. Additionally, data that requires manual entry or scraping (e.g., SEC filings) should be cleaned to ensure accuracy.

When data cleaning is applied only to historical data, but real-time data feeds are processed differently, discrepancies may occur, potentially undermining the reliability of your application and analysis. This issue arises because the cleaned historical data might not align with real-time data due to variations in the treatment of anomalies or errors.

Seen from a different angle, it's even more effective to delegate the identification and handling of “anomalies” to downstream consumers, such as trading units. In many cases, these anomalies represent real market behavior that convey useful information. Moreover, data errors and outliers may serve as input to incorporate within your trading strategies to adapt them to handle issues like market disruptions and trading interruption events (e.g., gateway failures and failovers, trading and regulatory halts, position limit breaches, sequence gaps, circuit breakers, and order book failures).⁶

Generally, there are three types of actions used to clean financial data:

Deletion

Low-quality records are deleted from the dataset. Examples include erroneous quotes, invalid prices, duplicate transactions, etc. When performing deletion, make sure to consider its impact on the analytical integrity and consistency of the data.

Correction

Low-quality records are replaced with their correct values. For example, a negative price is replaced with a positive one. Corrections need to be based on well-thought-out assumptions about the data (e.g., a price cannot be negative). In some instances, corrections may require notifying the entity that submitted incorrect data to resubmit it in the proper format (e.g., reporting financial data that doesn't conform to a financial messaging standard).

Enrichment

New fields are added to assist in detecting or mitigating the impact of low-quality records. This approach is useful when errors are difficult to detect, allowing the final data consumer to decide how to handle them. For example, if outlier detection is complex, a statistical model can generate an outlier probability for each record, which is then stored in a new column.

When performing data cleaning, it is important to keep in mind the two data governance principles of lineage and ownership (which we discussed in [Chapter 5](#)). Data lineage eliminates the possibility of data cleaning becoming a mystery box by guaranteeing visibility of the cleaning steps, decisions, and rules. Similarly, data ownership ensures that only the data owners have the authority to determine and implement data cleaning procedures, which makes assigning accountability for the data quality process more straightforward.

Data adjustments

Data adjustments are transformations applied to the data to account for specific data characteristics, events, and rules, or to produce more informative or analytically desired features.

In finance, the most frequently applied adjustment concerns [corporate actions](#). Such actions refer to important decisions made by companies that can significantly impact their stock value. The two most common types of corporate actions are *stock splits* and *dividend distributions*. A stock split involves the issuance of several new stocks for each existing stock to increase liquidity and make it more affordable for investors to buy the company's shares. For example, a 1:2 split means that each stock will be split in two, so if an investor holds 10 stocks, after the split they will end up with 20 stocks. A stock split does not change the total market capitalization of the company, but it impacts the stock price. To adjust for a split, the price of the stock needs to be divided by the split ratio. For example, if the price of a stock is \$400 and a stock split of 1:2 takes place, the new stock price is \$200.

Similarly, a dividend distribution event happens when a company decides to distribute part of its earnings to its shareholders. In such a case, the stock price needs to be adjusted to take into account the paid dividends. The standard approach to dividend adjustment consists of subtracting the dividend amount per share from the stock price. For example, if a com-

pany announces a dividend distribution equal to \$1 per share, and the stock price is \$11, the dividend-adjusted price will be $11 - 1 = \$10$.

When working with stock price data, pay particular attention to corporate action adjustments. First, determine whether the data source you are using incorporates these adjustments. Second, if the data is unadjusted, ensure this is clearly communicated and documented, and consider sourcing corporate action data separately. Some data sources, such as [CRSP US Stock Databases](#), provide both stock price data and corporate actions information. There are also specialized sources for corporate actions, such as S&P's [Managed Corporate Actions](#), LSEG's [Equity Corporate Actions](#), and NYSE's [Corporate Actions](#), among many others.

Another common technique is [calendar adjustment](#), which modifies a financial time series to remove calendar effects. For instance, the number of working days in a month can vary each year due to holidays, making it difficult to compare total production across months. One solution is to adjust the dataset by calculating the monthly average or considering daily figures.

Data standardization

Data standardization is a critical transformation step that seeks to store and format data according to a predefined set of conventions and standards. Examples of standardizations applied to financial data include the following:

- Date and time are formatted using the [ISO 8601—Date and Time Format standard](#), which follows the YYYY-MM-DD convention.
- Country names are represented following the [ISO 3166—Country Codes standard](#) (e.g., US for the United States of America).
- Currencies are represented using the [ISO 4217—Currency Codes standard](#) (e.g., USD for US dollars).
- Table and column names are lowercase, and spaces are replaced with an underscore (e.g., first_name).
- Monetary values are [standardized](#) to use one numerical format (e.g., EUR 10 million, EUR 10,000,000, or EUR 10000000).
- Monetary values are rounded off to a specified level of precision to ensure consistency and comparability across financial records.
- Standardized identifiers are used for financial instruments, such as ISIN or FIGI.

These standardizations help maintain data quality, facilitate data integration, and ensure compatibility across different financial systems and applications.

Rounding is one of the most common transformations applied to financial data. Interestingly, despite its apparent simplicity, rounding might actually require careful consideration. First of all, [a large variety of rounding algorithms exist](#), each tailored to a specific use case. Certain methods, such as *Bankers' Rounding*, are quite common in finance. This method aims to evenly distribute rounding errors, thereby reducing potential bias in the data. For example, 2.5 would be rounded to 2, while 3.5 would be rounded to 4.

Furthermore, deciding on the rounding precision might depend on the specific financial variable and accepted market practices. For example, a common practice is to set the rounding precision in line with the *Minimum Price Increment* (MPI), also called minimum tick size. This represents the smallest possible price change in a financial instrument's price. MPIs vary depending on the asset class, market regulations, and the specific trading venue. For example, in the United States, the MPI is typically \$0.01 for stocks priced above \$1, while for stocks priced below \$1, it can be \$0.0001. In Forex markets, the term *Point in Percentage* (PIP) is used to denote the smallest amount by which the quote can change. A typical PIP is equal to one basis point, or 0.0001, but it can vary among trading venues and Forex brokers depending on the currency pair and the lot size traded. Following this logic, you might decide to set the rounding precision equal to the MPI; for example, an MPI of 0.0001 means a decimal precision of 4.

In addition, rounding might depend on whether you want to round the decimal places or significant figures. To illustrate, let's consider a currency pair in which one currency has a much higher value than the other, say 0.00247839. It is possible to round this number to five decimal places, in which case it becomes 0.00248. But if we want to round to five significant figures, we would get 0.0024784.

To further explore this topic, I highly recommend the book by Brian Buzzelli, [Data Quality Engineering in Financial Services](#) (O'Reilly, 2022).

In today's financial markets, a key data engineering challenge is harmonizing and standardizing the diverse data formats, structures, and types from various sources. Whether it's market data, transactions, or external feeds, achieving consistency and interoperability is essential for accurate analysis and AI-driven insights. To give a recent example of how markets are approaching this problem, J.P. Morgan's Fusion, a cloud-native data platform for institutional investors, launched data containers in May 2024. These containers use a common semantic layer to model and normalize data across multiple providers, sources, and formats—giving investors a consistent, integrated view of their data.

Data filtering

Data filtering is an analytical transformation step whereby a financial dataset is examined to exclude or rearrange its records according to pre-defined criteria. In other words, data filtering applies a filter to a dataset, transforming it into a new dataset based on the filter criteria.

A wide range of data filters are used in finance. Here are a few examples:

Company filter

This consists of excluding companies from a financial dataset that don't satisfy certain conditions. For instance, an [article written by Priyank Gandhi and Hanno Lustig](#) details a study using the CRSP dataset, excluding inactive firms (legal entities without business activities) and firms incorporated outside the US to ensure a uniform regulatory regime.

Calendar filter

This filter is used to align accounting standards across firms in a dataset. For example, in an [article written by Laura Xiaolei Liu and Lu Zhang](#), they excluded firms that do not have a December fiscal year-end.

Liquidity filter

This filter is used to ensure that the securities included in a data sample have at least a certain number of active trading days during a specific time interval. In one [research paper](#), the authors applied a filter to exclude stocks with less than two hundred days of active trading in a year to have a sample with enough data for computing liquidity measures. Similarly, when working with option data, it is common to filter out options whose expiration falls outside a given interval (e.g., between 10 and 400 days) as such options might behave erratically near expiry due to liquidity features (the article ["The Puzzle of Index Option Returns"](#) provides a good example).

Size filter

This filter excludes firms whose market capitalization is below or above a certain value, for example, when studying a specific segment of the market. In one [study](#), the authors excluded microcap stocks, defined at the fifth percentile of market capitalization within each country. This was done because minicap stocks often suffer from stale prices and volumes due to market illiquidity as well as their negligible economic significance (<0.04%) on the overall market value.

Coverage filter

This filter excludes firms that do not have enough observations in the dataset. This is often done to reduce the impact of noise generated by such observations. For example, the authors of a [study on factors that drive global stock returns](#) applied a similar filter by re-

quiring a stock to have at least 12 monthly observations within the sample period to be considered for inclusion in the study.

Sector filter

This involves including entities belonging to a particular sector, market segment, industry, or subindustry. This filter is applied because certain sectors can be subject to special regulation or have different asset and investment structures. For example, in their seminal work, *The Cross-Section of Expected Stock Returns* (Wiley), Fama and French conducted their analysis by excluding financial firms, as high leverage for these firms does not indicate the same thing for nonfinancial firms, i.e., distress.

When applying data filters, it's essential to understand their purpose and the potential impact on data quality and integrity. Improper filtering can introduce biases, such as nonrepresentative sample bias or imbalanced datasets. As a financial data engineer, you should discuss these considerations with the end users of the data. Additionally, ensure that filters are applied correctly and do not modify or delete the original data. If a filtered dataset is needed, create a new table for it.

Feature engineering

Feature engineering is an advanced analytical transformation step used to extract new features from raw data to support statistical and machine learning modeling. Feature engineering is often necessary when existing features do not adequately represent the problem at hand. A feature is a variable or measurable quantity used as input in various modeling tasks. It can be numeric, categorical, binary, or text based. In database terms, a feature can be thought of as a new column in a given table. It is a well-known empirical fact that [the performance of machine learning systems is heavily dependent on the feature representation of the input data.](#)

The practice of feature engineering is quite flexible; data analysts and machine learning experts have the freedom to experiment with and derive novel features from a given dataset to represent different aspects of the data that are relevant to the situation at hand.

Feature engineering can be data driven (e.g., based on statistical correlations or patterns in the data) or domain driven (e.g., based on a financial theory). Moreover, feature engineering requirements may vary based on the type of data being analyzed. For example, there is feature engineering for text data, visual data, time series data, graph data, and stream data. A full account of these techniques is beyond the scope of this book, and for this, I recommend the excellent reference *Feature Engineering for Machine Learning and Data Analytics*, edited by Guozhu Dong and Huan Liu (CRC Press, 2018). Nevertheless, to give it a minimal treatment, let's go through some examples of feature engineering.

The most general techniques for feature engineering include the following:

Normalization

Rescaling the data to fit within a predefined range, e.g., [0–1]. This is often done to prevent some features from having a dominant impact during model training.

Scaling

Rescaling the data to have a similar scale, such as a standard deviation of 1, to ensure a model considers all features equally. Common scaling techniques include *min-max scaling* and *standard scaling*.

Encoding

Transforming a feature from categorical to numerical representation; e.g., female is 1 and male is 2. Common encoding techniques include *one-hot encoding* and *label encoding*.

Dimensionality reduction

Transforming a set of features from a high-dimensional space into a lower-dimensional one. Examples include *principal component analysis* and *t-SNE*.

Embedding

A technique used to create numerical representations of complex real-world objects that machine learning systems can use in model training. As an illustrative example, an embedding might take an image, audio, text, or a graph and convert it into multidimensional numerical representations known as vectors. Using vectors, machine learning systems can efficiently process input data and identify similarities among different data items (e.g., two similar images). A special type of database, called a [vector database](#), has been developed to allow ML-driven applications to store and retrieve vector datasets.

In finance, a wide range of domain-specific feature engineering techniques are often applied. For example, when conducting financial time series analysis, it is common to perform steps such as *de-trending* and *de-seasonalization*.⁷ De-trending is the process of removing a trend cycle from the data. A trend cycle refers to a consistent increase or decrease of a financial time series over time. On the other hand, de-seasonalization removes seasonal patterns from the time series. A seasonal pattern refers to a specific event in the time series that occurs with a fixed and known frequency such as daily, weekly, every January, etc.⁸ By applying de-trending and de-seasonalization, new features are created.

Another widely used technique in finance is stationary differentiation. This involves calculating the difference between each consecutive pair of observations over time (i.e., $x_t - x_{t-1}$, $x_{t-1} - x_{t-2}$, ...). It is commonly used to transform a non-stationary financial time series into a stationary one. In simple words, a stationary time series is a series whose statistical properties do not change over time. Stationarity is a desired property, as it makes it simpler to perform data analysis using classical statistical

methods (e.g., inferential statistics). Converting a price time series to a return series is one such application of differentiation that financial analysts often perform. This is done by taking the percentage difference between two consecutive prices (i.e., $(x_t - x_{t-1})/x_{t-1}$).⁹

In addition, a popular feature engineering technique applied by financial analysts is log transformation, where each value x is replaced with $\log(x)$. Expressing data on a logarithmic scale can be helpful for analytical purposes. For example, if a price time series is expressed in a log scale, the difference between two log prices approximates the percentage change (i.e., $\log(p_t) - \log(p_{t-1}) \sim (p_t - p_{t-1})/p_{t-1}$). Furthermore, the log transformation is frequently used to transform skewed financial data to conform to normality, which is a desired feature in financial analysis.¹⁰

Finally, an advanced form of feature engineering in finance is the creation of factors. In financial investment literature, “factors” refer to common asset characteristics that explain variations in returns and risks across stocks, bonds, and other assets. They can explain why certain assets go up or down at the same time and why certain assets may yield higher returns compared to others.

There are two main types of factors: macroeconomic factors and style factors. Macroeconomic factors capture risks that affect broad segments of the financial markets and impact multiple asset classes simultaneously. Examples include interest rate (the impact of interest rate changes), inflation (the effect of price level changes), and economic growth (variations in the business cycle). Style factors, on the other hand, explain returns and risks within individual assets or asset classes. Examples include value (assets undervalued relative to their fundamentals), momentum (assets with upward price trends), low volatility (assets with a lower risk profile), quality (assets of financially robust companies), and growth (assets or companies with strong earnings growth potential). These factors are engineered features derived from fundamental or market data to enhance portfolio returns and manage risk.¹¹

Advanced analytical computations

An advanced type of data transformation may involve the computation of one or more quantities based on an algorithm or a model. Generally speaking, financial data engineers seldom perform scientific modeling or develop machine learning algorithms. However, with the rise of data products, data platforms, and analytics engineering, there are contexts where financial data engineers, particularly those with interdisciplinary backgrounds, might become involved in the modeling process.

Examples of financial applications that require advanced computations include the following:

- Algorithmic trading
- Financial recommender systems (e.g., robo advisors)
- Fraud detection

- Anti-money laundering
- Named entity recognition
- Entity resolution
- Optical character recognition (e.g., recognizing the digits on a credit card image)

To develop such systems, financial data engineers may need to handle tasks such as data collection, cleaning, quality assurance, and control checks. They are also responsible for selecting suitable DSMs and DSSs tailored to each application's requirements. Furthermore, financial data engineers may be involved in building machine learning pipelines, deploying them in production, and collecting metrics and model artifacts.

Transformation Patterns

The next stage after defining your transformation operations is to design and implement your transformation patterns. A transformation pattern defines how a given data infrastructure handles and performs transformations. To illustrate the concept, let's have a look at a few examples.

Batch versus streaming transformations

In batch transformation, data is divided into discrete chunks (batches) that undergo separate processing. A data chunk can be defined in various ways. For instance, if data is received in file formats like CSV or JSON, each file can be treated as a chunk and processed individually. Alternatively, if the files can be grouped based on specific criteria (e.g., by date), a chunk might encompass all files belonging to a particular group (e.g., all files for a given day). Batch file transformation is a common practice in finance, especially when data is delivered through files. Examples include financial data vendors, which distribute their data in daily, weekly, and monthly file batches, and financial reporting, which is often done via files submitted to the regulator's data infrastructure.

Batch transformations are often used with scheduled data arrival processes, which we discussed in [Chapter 7](#). In this case, the batch transformation is scheduled to run on a predefined interval (e.g., hourly, daily, weekly). Alternatively, a batch can be transformed once it is complete. For instance, a file batch with a maximum size of 10 will be transformed when it contains 10 files.

When data needs to be transformed as soon as it arrives, and not wait for a batch to complete, then a *streaming transformation* is used. In this case, there is no fixed schedule for data transformation; it runs continuously and processes data immediately upon its arrival. Streaming transformations are most commonly used with event-driven arrival processes, which were also covered in [Chapter 7](#). This is the case, for example, with real-time financial applications such as payments, fraud detection, price feeds, and news. In these applications, the system needs to respond with minimum latency and, therefore, cannot wait for a batch to complete.

Streaming transformations are often performed on independent data. For instance, in a scenario where a bank allows clients to submit loan applications online, each application can be processed immediately upon submission, without the need to wait for other applications to form a batch.

In certain scenarios, data arrival may occur in real time or be event driven, yet streaming transformation may not be necessary. For instance, when bank clients fill out a questionnaire to evaluate their financial literacy, the data does not require immediate processing. Instead, it can be processed in batches at regular intervals or as soon as a batch is complete. Using batch transformation to process real-time or event-driven data is a common practice employed to handle massive volumes of data or reduce the costs of continuous monitoring for new data arrivals.

Figure 9-3 illustrates how batch and streaming transformations work. In batch transformation (top), data files arrive and get ingested into a data lake. Subsequently, files are grouped in separate batches based on date. After that, each batch is transformed separately in the transformation layer. Once transformed, the data is stored in a given target location, such as a warehouse. In the streaming transformation case (bottom), data arrives in JSON format. Each JSON gets immediately ingested into a message broker. Subsequently, each message is processed as soon as possible in the transformation layer and stored in the final data warehouse.

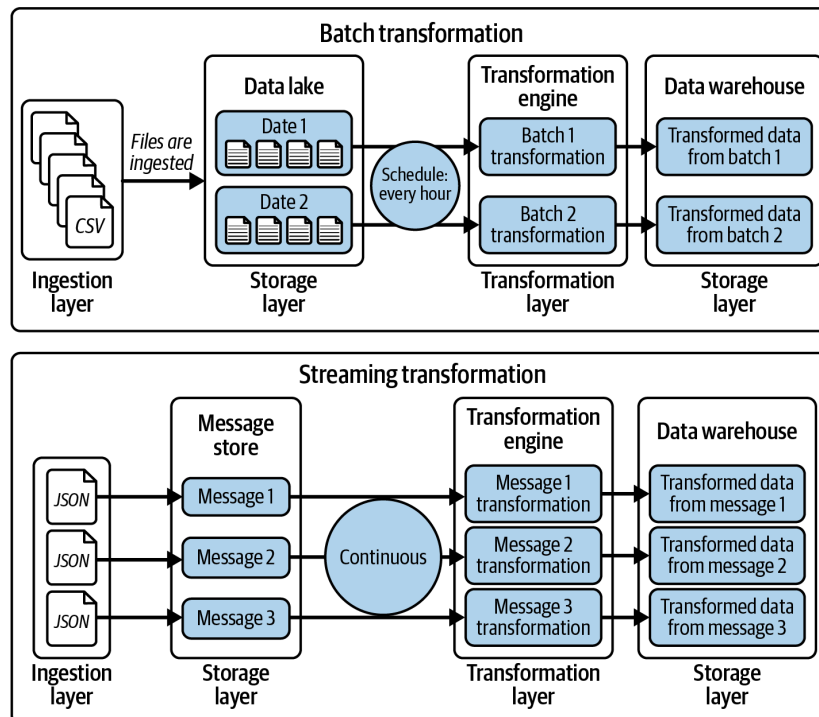


Figure 9-3. Standard batch and streaming transformation patterns

Memory-based versus disk-based transformations

A given data transformation, whether batch or streaming, can be processed either completely in memory or involving intermediary disk persistence steps. The best way to show the difference is with an illustrative example, as shown in **Figure 9-4**. A basic disk-based transformation is illustrated in the figure's upper part. In this setting, files are first ingested

into a data lake. Subsequently, the transformation layer applies two transformation iterations on the files (e.g., cleaning + feature engineering) and saves the final results in a data warehouse. Note, however, that between the two iterations, the transformation layer had to store intermediary results back to the data lake and read it again for the second iteration. In the lower part of the figure, a memory-based transformation does the same thing, but instead of saving intermediary results to the data lake, it keeps it in memory and passes it as such to the next interaction.

Disk-based transformation

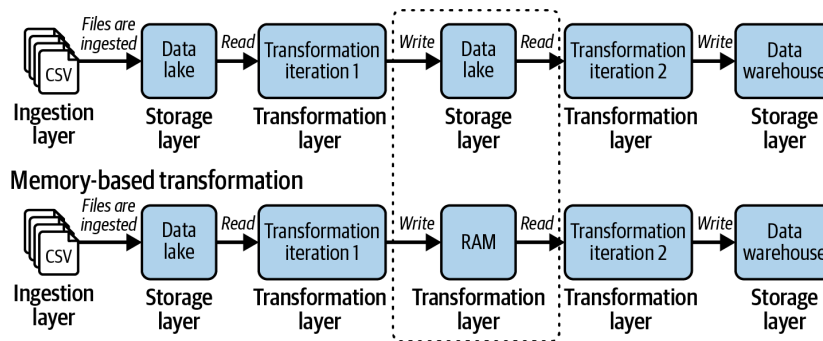


Figure 9-4. Disk-based versus memory-based data transformations

You might be wondering by now why this is necessary. The answer lies in the significant difference in data access speed between RAM and disk (random disk access, to be precise). This is the reason why in-memory software solutions for data storage and processing are quite popular.¹²

Many financial applications rely on memory-based data transformations, especially in time-critical scenarios like trading and fraud detection. For instance, in high-frequency and algorithmic trading, real-time data from feeds is processed directly in memory for immediate action. If data were to be stored on disk first, then the speed advantage would be lost.

NOTE

Disk-based access modes don't all operate slowly. When compared to RAM access, random disk access—where data is retrieved at random locations on disk—is especially sluggish. Sequential disk access, on the other hand, is very quick since data records are retrieved in a predetermined order. Sequential disk access is leveraged by technologies like [Apache Kafka](#) in order to achieve high-performance data read/write operations.

When discussing data computing frameworks, it is difficult to avoid mentioning *Apache Spark*, a unified framework for large-scale data analytics. A bit of history is important to help understand the emergence of Spark. Early efforts at processing large data volumes led to the development of *Apache Hadoop*, a rich ecosystem of open source tools designed for managing multinode clusters and processing massive datasets. Hadoop's ecosystem incorporates two key components: the Hadoop Distributed File System (which we discussed in [Chapter 8](#)) for distributed storage with high-throughput access to data, and MapReduce for large-scale parallel data processing. Crucially, MapReduce's internal design processes data on disk, which was a major downside in terms of performance. In response, Apache Spark emerged as the next evolution in the Hadoop ecosystem. Compared to Hadoop MapReduce, Spark has demonstrated [significant performance improvements](#).

One of Spark's primary advantages is its native ability to perform computations in memory. While Spark can handle computations on disk when data exceeds memory capacity, data engineers typically configure sufficiently large Spark clusters to enable in-memory processing. Spark's core memory data abstraction is known as the *Resilient Distributed Dataset* (RDD), which is immutable and can be distributed across nodes within a Spark cluster.

Spark is a rich framework that combines Spark Core (execution engine), Spark SQL (for structured data querying and processing), Spark Streaming (for streaming processing), the Spark machine learning library, MLlib (for building machine learning models), and GraphX (for graph data processing).

Furthermore, Spark seamlessly integrates with several programming languages such as Scala, Java, Python, and R, allowing developers to write Spark programs in any of these languages. Among data engineers, the Python API known as PySpark is particularly popular and widely used.

Spark can be deployed on premises and on the cloud (e.g., using a cluster of Amazon EC2 instances). Managed cloud solutions such as Amazon EMR allow users to configure a Spark cluster without the need to deal with cluster and node configurations. Additionally, some cloud-based AI and analytics frameworks are powered by Apache Spark, for example, Databricks and Azure Synapse Analytics.

Apache Spark has multiple applications in finance. For example, it can be used to perform fast data transformations that involve massive financial datasets, such as high-frequency trades and quotes. Fraud detection is another use. For an illustrative architecture design, consider starting with a Spark ML model, such as logistic regression, designed for fraud detection. Data begins by entering an event stream through a message broker like Kafka, continues to Spark Streaming for real-time processing, and finally undergoes fraud verification using the deployed Spark ML model.¹³

Another scenario that requires deciding between in-memory and disk-based transformations involves choosing whether to perform transformations dynamically in memory or precomputing and storing them in a database beforehand. This can be the case, for example, when performing feature engineering. Doing feature engineering dynamically in memory can be advantageous for large and changing datasets, enabling real-time processing without the overhead of storing and managing precomputed features. This approach allows flexibility in adapting to changing data requirements and quality, but it may require substantial computational resources (e.g., RAM) and execution time. On the other hand, precomputing and persisting features in a DSS can enhance performance by reducing computation time and memory consumption during model training and inference, which is optimal in scenarios involving expensive queries and complex feature engineering steps. Furthermore, this approach ensures consistency and reproducibility of features across different stages of model development and deployment. Therefore, the choice should be guided by balancing computational efficiency, reproducibility, traceability, data freshness, and scalability needs.

Full versus incremental data transformations

In a full data transformation pattern, the entire dataset (or its complete history) is transformed in one go, regardless of whether parts of it have been modified. This approach is typically used with small datasets that don't change frequently. Its main advantages are simplicity (no need for sophisticated logic to process parts of the data), consistency (the entire dataset is processed uniformly), and simpler error handling (if an error occurs during transformation, it's easy to detect since the entire operation will fail). This approach has drawbacks such as being resource intensive (especially with large datasets), limited scalability (if the dataset becomes large, processing it as a whole can become impractical or time-consuming), and potential latency issues (which may affect time-critical applications).

Alternatively, incremental data transformation involves transforming only new or updated data, rather than the entire dataset. This approach is commonly used with large datasets or applications that continuously generate and update data. Its main advantages include resource efficiency, scalability, low latency, and reduced costs. Note, however, that incremental data transformation may introduce additional complexity, as it requires implementing a change detection mechanism (e.g., Change Data Capture), data ingestion logic to handle only new or updated records, and data processing logic capable of inserting or updating records without overwriting existing ones.

Change Data Capture (CDC) is an essential concept in data engineering. It refers to the capability of a data infrastructure to detect changes—such as inserts, updates, or deletes—in an upstream data source and propagate them across downstream systems that consume the data. A common scenario is the propagation of changes from an operational database to an analytical system such as a warehouse or a data lake. CDC is extensively used to ensure data consistency and integrity across various systems, support up-to-date real-time analytics, and help organizations migrate from on-premises to the cloud.

A CDC mechanism can operate in either a push or pull manner. In a push-based CDC, the source data storage system sends data changes to downstream applications. In a pull-based CDC, downstream applications regularly poll the source data storage system to retrieve data changes.

Several methods exist for implementing CDC. A straightforward approach involves adding a timestamp column that records the time of the latest changes. Downstream systems can then capture updates by querying data with timestamps greater than the last extracted timestamp. The main disadvantage of this method is that it may not effectively capture deletes.

Another common method relies on database triggers. A trigger is a stored procedure in a database that executes a specific function when certain events, such as inserts, updates, or deletes, occur. Triggers can propagate changes immediately but may add additional load to the source database. Some data storage solutions, such as [MongoDB](#), implement a separate trigger layer that scales independently of the database server.

Finally, a highly reliable CDC approach involves using database logs. To ensure durability and consistency, many database systems log all changes into a transaction log before persisting them to the database. Logs are quite reliable as they capture all changes made to the data, along with metadata about who made the changes and when. The main challenge with this approach is the potential complexity of setting it up and maintaining it.

Computational Requirements

When designing the transformation layer, you must assess and plan your computational requirements. The main factors to consider are computing environments and performance.

Computational performance

Having a framework that outlines performance requirements and optimization strategies is essential for a reliable transformation layer. This framework should include features such as computational speed, throughput, efficiency, and scalability.

Computational speed

Speed is a critical requirement for financial data transformations. It is typically measured by the time difference between the start and end time of a data transformation execution. A related and more specific concept is *latency*, which denotes the time it takes for a signal or message to travel over a network to its destination and receive a response back from it. Common latency metrics include the *average latency*, which is the mean time it takes for two systems to exchange messages, and *latency jitter*, which is the variation in latencies around the average value.

Throughout the rest of this section, I will use the term computational speed to describe the difference between the expected and the actual processing time of a given data transformation. This presupposes that you, as a financial engineer, should discuss and determine the ideal execution time for the many financial data transformations and keep an eye on your infrastructure to make sure it performs up to par. This can be linked to the data timeliness dimension that you learned about in [Chapter 5](#).

NOTE

In general, the higher the computational speed you aim for, the more technically challenging it becomes. It is simpler to reduce computation time from hours to minutes than from seconds to microseconds. For this reason, treat it as an economic problem by estimating the marginal gain from every unit of improvement in computational speed. Certain types of financial data transformations (e.g., monthly price extractions) may not require significant computational speed.

To guarantee fast data transformations, a common practice involves the definition of a Service-Level Agreement (SLA) that outlines the average duration of data transformations (e.g., five seconds) or the time by which a specific data transformation should have been finished (every Friday at 10 a.m.).

Speed in financial markets comes with its own set of risks. For instance, the drive toward instant payment and settlement processes increases exposure to fraud risks, as the system has limited time to analyze, detect, and prevent fraudulent activities and ensure compliance. Moreover, this can expose participants to liquidity, market, and credit risks. For example, during settlement, financial institutions must ensure they have sufficient liquidity to handle instant payments, particularly during peak times. Instant payments also expose market participants to credit risk if counterparties fail to settle transactions promptly.

High-frequency trading is another area where speed is key, with rapid data access and quick decision-making being critical elements. However, relying on real-time market data for fast trading might, in some cases, be risky because the data may contain errors or noise, leaving the trading system with limited time to properly assess the quality of the data.

Furthermore, fast trading systems like high-frequency and algorithmic trading can amplify market volatility, particularly during periods of stress. In addition, the pursuit of speed has contributed to events like flash crashes, where market prices drop sharply and recover within minutes. The reliance on sophisticated algorithms and high-speed trading infrastructure increases the risk of technical failures, such as software bugs, hardware malfunctions, or connectivity issues.

Fast order execution in trading poses challenges for regulatory compliance as well. For instance, rapid order cancellations or modifications by high-frequency traders can strain market surveillance systems responsible for upholding fair market practices and preventing market manipulation.

These examples highlight the distinctive characteristics of data engineering in finance compared to other industries. While speed may not pose major risks in certain sectors, in finance, rapid transactions and decisions can introduce substantial risks that must be carefully considered when designing financial data infrastructures.

Once an SLA is defined, the next step would be to compare the actual data transformation time against the SLA specification. Should the SLA be violated, you, as a financial data engineer, need to understand the main causes and propose potential solutions. Factors that might impact data transformation execution time include the following:

- Low-quality data that requires more steps to clean
- Poor querying patterns that take more time than necessary
- Dispersed data that requires multiple queries to collect
- A wrong data storage model (e.g., using a data lake instead of a data warehouse for structured data)
- Large batch jobs
- Limitations on the database side (e.g., max concurrent connections)

- Poor database design (e.g., missing or bad indexes)
- Too much complexity in the transformation logic
- Insufficient compute resources (e.g., low RAM or CPU)
- Shared resources that need to respond to requests coming from a variety of applications
- Bad queuing strategy (e.g., jobs are not ordered and executed in the right priority, big or time-consuming batch jobs run before small interactive requests, lack of asynchronous or parallel execution capabilities)
- Wrong transformation patterns (e.g., executing data transformations with synchronous blocking can slow down performance compared to asynchronous event-driven or streaming solutions; more on this in [Chapter 11](#))
- Network issues
- Too many processing layers
- Inefficient data distribution among data centers (data is far from the processing engine)

Ideally, you should anticipate and test for such issues in advance. As discussed in [Chapter 8](#), changing your data storage system is a costly operation that should be avoided. Similarly, once your data transformation layer is defined and implemented, you don't want to migrate or alter it.

Throughput

In data-intensive applications, an important performance measure is throughput. This refers to the amount of data that a system can process in a given time interval. Applications designed for high throughput emphasize the total amount of work that can be done in a given period of time. This is often measured using metrics such as bits per second (bit/s or bps), megabytes per second, or records per second.

In today's financial markets, especially trading systems, high throughput is essential due to the continuous influx of a massive number of orders that require immediate processing.¹⁴ From an engineering perspective, it's crucial to define the desired level of throughput clearly. Simply aiming for "high throughput" is not specific enough. As such, it's essential to define a target throughput level through discussions with your business team and stakeholders.

The level of throughput may depend on a variety of factors, including the characteristics of the physical and networking infrastructure, the size and type of the data being processed in each request, and the type of operation being performed. For instance, when optimizing database read/write throughput, it is important to keep in mind that reading and writing have different internal mechanisms.¹⁵

Computational efficiency

The term computational efficiency is used to describe how well data transformations make use of available resources. An efficient data trans-

formation minimizes resource consumption, which is increasingly important given the current emphasis on the environmental impact of data computations, such as carbon footprint.

The term *algorithmic efficiency* is often used in this context to measure the computational resources that an algorithm uses. Generally speaking, the more time and space an algorithm consumes, the less efficient it is. Interestingly, in 2020, a group of researchers from MIT's Computer Science and Artificial Intelligence Laboratory (CSAIL) presented evidence showing that [data-intensive tasks benefit more from algorithmic efficiency than hardware improvement](#).

Finance has a lot of computationally expensive problems. Examples include risk valuation, derivative pricing, stress testing, scenario analysis, and Credit Value Adjustments (CVAs). Such problems are computationally expensive due to their nonlinear and high-dimensional nature, which results in a massive number of computations that need to be performed. I won't go into the details of these problems, but for those interested, I suggest reading the S&P Global blog entry [“Accelerating CVA Calculations Using Quasi Monte Carlo Methods”](#) on accelerating CVA calculations that illustrates the complexity of computing CVAs.

To address such computational challenges, the industry has responded with two streams of improvement: software-side and hardware-side improvements. Software-side improvements often entail the development of more efficient algorithms and computational models. On the hardware side, financial markets have expressed interest in what is called *High-Performance Computing* (HPC). HPC refers to the practice of aggregating and connecting multiple computing resources to solve complex computation problems efficiently. HPC isn't just one technology, but rather [a model to build powerful computing environments](#).

Designing an HPC cluster leverages several concepts and technologies such as parallel computing, distributed computing, virtualization, Central Processing Units (CPUs), Graphics Processing Units (GPUs), in-memory computation, networking, and many more.

An HPC cluster can be implemented in a variety of ways. For example, an Apache Spark cluster can be considered an HPC. A cluster of connected EC2 instances is also an HPC environment. More complex forms of HPC clusters combine heterogeneous types of machines that serve different purposes (e.g., compute-optimized with high CPU count, memory-optimized, GPU-accelerated). Some organizations develop custom-built HPC supercomputers, highly optimized for superior speed and performance compared to standard computing systems. Supercomputers are often benchmarked using the [floating-point operations per second \(FLOPS\) measure](#).

For more on HPC in finance, see Michael Alan Howarth Dempster, Juho Kannianen, John Keane, and Erik Vynckier, eds., *High-Performance Computing in Finance: Problems, Methods, and Solutions* (CRC Press, 2018).

Scalability

Computational performance is very often dependent on the scalability features of the underlying infrastructure. When there is an increase in the number of data transformations or sudden peaks in workload, the computational capacity needs to scale proportionally to efficiently manage the increased load.

Taking a cloud context as a reference, scalability can be achieved by either adding more resources (e.g., additional EC2 instances or cloud functions) or by upgrading the capacity of existing resources (e.g., replacing smaller EC2 instances with larger ones). Scalability can be implemented following different strategies, such as the following:

Manual scaling

Where you manually scale existing resources to meet new demands

Dynamic scaling

Where you configure an autoscaling policy to automatically provision more resources in reaction to larger demands (e.g., if total CPU usage > 90% → provision X resources)

Scheduled scaling

When you configure an autoscaling policy to provision more resources on a given schedule (e.g., every Wednesday between 9 a.m. and 4 p.m., provision 200 more EC2 instances)

Predictive scaling

Where a machine learning model is used to predict and provision resources based on historical usage patterns

Scalability requires careful consideration. In particular, when planning the scaling requirements for the transformation layer, it's important to also assess the scalability features of the storage layer. For example, if you have a max concurrency limit on the data storage layer (e.g., 500 concurrent requests), it's essential to properly manage the number of concurrent requests arriving from the transformation layer to avoid overloading the storage layer. Ideally, the design of both the storage and transformation layers should be iterative, ensuring compatibility in scalability features to achieve optimal performance.

Computing environments

Once you have identified your computational requirements, the next step is choosing a computing environment that aligns with these specifications. Such an environment typically comprises three key components: software (e.g., operating system, programming language, libraries, frameworks), hardware (e.g., storage, RAM, CPU, GPU), and networking (e.g., TCP/IP, VPC, etc.).

There is a large variety of computing environments that can be configured to run the transformation layer tasks. Traditionally, many financial institutions, in particular banks, have [relied on mainframe computing environments](#) that run on languages such as Common Business Oriented Language (COBOL). However, modern financial applications are leveraging open source technologies and the cloud as a viable and simpler alternative.

If we choose to leverage cloud infrastructure, there are several approaches to consider when setting up the computing environment for the data transformation layer. The most flexible strategy is infrastructure-as-a-service (IaaS), where you provision and configure several computing machines to perform the data transformations. In this setting, you will be responsible for installing the required programming languages (e.g., Python, Java, Go, etc.), configuring security policies (e.g., ingress and egress rules), and installing all necessary packages (e.g., PySpark, Apache Airflow, etc.). The main advantage of IaaS is the extra control it offers over environment configuration. However, managing, configuring, and securing your instances under IaaS can potentially lead to a waste of time and effort.

As an alternative to IaaS, cloud providers offer several managed services that alleviate the need to configure and maintain the compute instances. In such settings, the user interacts with a declarative interface where they define the desired configuration of their environment. The cloud provider takes responsibility for provisioning, scaling, and managing the underlying infrastructure, offering greater convenience and often reducing administrative overhead for users compared to the IaaS model. For example, AWS offers *Managed Workflows for Apache Airflow* (MWAA), a managed service for running the data workflow management solution Apache Airflow (which we will discuss in [“Extract-Transform-Load Workflows”](#)).

Another popular choice for cloud-based computing environments is serverless cloud functions, which allow users to deploy and run code in a variety of languages without provisioning or managing servers. Cloud functions are ideal for short-time data transformation operations (several seconds or minutes). Examples include AWS Lambda and Google Cloud Functions (first and second generations). One of the main advantages of cloud functions is that they can be integrated with a wide range of other services and handle event-based workloads. For example, it is possible to configure a cloud function to run upon the arrival of a file in storage (e.g., Amazon S3), upon the arrival of a message in a queue (e.g., Amazon Kinesis), or with database updates (e.g., Amazon DynamoDB).

The Financial Industry Regulatory Authority (FINRA) is a US nongovernmental organization responsible for protecting investors and ensuring market integrity by overseeing and regulating broker-dealers. To monitor the trading practices of member firms, FINRA has an integrated audit trail system of orders, quotes, and trade events for National Market System (NMS) stocks and over-the-counter (OTC) equities. To record such data, FINRA's audit trail relies on the Order Audit Trail System (OATS). Using OATS data, along with other sources of market and reference data, FINRA is able to reconstruct the lifecycle of a trade—from origination through completion or cancellation—and monitor the practices of its members.

Member firms submit daily OATS data to FINRA, totaling more than 50,000 files each day. Upon receipt, FINRA verifies the data's completeness and proper formatting against more than 200 rules, processing up to half a trillion validations daily. To handle the significant and variable processing demands, FINRA needed a scalable, cost-efficient, and secure solution.

Three options were explored—Apache Ignite on Amazon EC2, Apache Spark on Amazon EMR, and AWS Lambda. AWS Lambda emerged as the optimal choice due to its scalability, efficient data partitioning, robust monitoring, high performance, cost-effectiveness, and minimal maintenance needs. This choice supported FINRA's goal of transitioning to a real-time processing model.

Security was a critical factor, and AWS met FINRA's stringent data-protection requirements, including encryption of data in transit and at rest. The new system was developed in three months, with data ingested into Amazon S3 via FTP and validated using AWS Lambda functions. A controller, running on Amazon EC2, manages data feeds into AWS Lambda and outgoing notifications, as well as external data sources like stock symbol reference files.

To ensure continuous operation and reduce processing time, the new architecture leverages AWS Lambda's data-caching abilities and uses Amazon SQS for input/output notifications.

Data Delivery

Once data has been transformed, the next step is to deliver it to the final consumers to extract actionable insights. Financial data engineers must determine who the ultimate data consumers are and understand their specific needs, and then create the appropriate mechanisms to deliver the data. The following two sections will provide a brief overview of this process.

Data Consumers

Any user, application, business unit, team, or system that makes use of data generated by their company's data infrastructure is considered a data consumer. Human data consumers can engage in the data engineering lifecycle to varying extents. For example, compliance officers and marketing teams often specify their data needs and rely on data engineers to handle the rest. In contrast, analysts and machine learning specialists may be more involved in defining the data source, type, and necessary transformations.

Data consumers may also differ in terms of their data governance duties and responsibilities. For instance, senior individuals in control of the data in a certain domain are known as data owners. In a bank, for instance, the finance director may be the owner of client financial data. A related role is that of the *data steward*, who is responsible for maintaining and guaranteeing the quality and consistency of data as defined by the data owner. Data custodians are in charge of protecting the data by adhering to the rules outlined in the data governance framework.

Financial data engineers, being the data producers, must ensure that data consumers have clear and well-defined expectations and requirements to facilitate effective data delivery. A reliable way to formalize such an agreement between data consumers and producers is through data contracts, which were discussed in [Chapter 5](#). A data contract can detail all consumer requirements such as the data type, fields, formats, constraints, conversions, SLA, and many more. Data contracts are often owned by the data owner or delegated to the data steward. With a data contract, one can be sure data producers know exactly what data consumers want, thus avoiding miscommunication issues around the data.

Delivery Mechanisms

Data can be delivered to its final consumers in a variety of ways. These include the following:

- Direct database access via a user interface (e.g., Snowflake UI, pgAdmin for PostgreSQL, Compass for MongoDB, etc.). This mechanism is often intended for people with basic SQL knowledge who want to conduct exploratory data analysis.
- Direct file access via a user interface (e.g., the S3 web interface). This is intended for easy interaction and sharing of the data.
- Programmatic access to databases and file repositories via APIs, JDBC (Java Database Connectivity), ODBC (Open Database Connectivity), and client libraries such as AWS Boto3. This delivery mechanism is widely used by applications, software engineers, and data engineers alike.
- Reports that contain essential summaries and aggregations of data used for supporting decision-making.
- Dashboard access that displays metric and summary data in a visual format, typically via a single web page. Dashboards are feature rich, and come with a variety of tools to visualize, explore, filter, compare,

zoom, and query the data in a user-friendly way. They are quite useful for stakeholders who don't have much expertise or the time to perform raw data querying and analysis.

- Email delivery.

An important thing to keep in mind when designing a delivery mechanism is the need to provide users with the means to search and find the data they are looking for. As more data gets generated and stored in various locations, it becomes harder for data consumers to know where to find what they are looking for. Some good practices can be adopted in this regard. For example, the final location of the data needs to be specified in the data contract. Additionally, a data catalog can be created as a central search engine that allows users to search and find the data they are seeking.

Summary

This chapter discussed the third layer in the financial data engineering lifecycle: the transformation and delivery layer. Key topics covered include data querying patterns, query optimization, transformation operations and patterns, data consumers, and delivery mechanisms. Throughout the chapter, these concepts are applied within the context of financial markets, providing practical examples and case studies for better insights.

One thing to keep in mind is that financial data transformations, along with their associated patterns and optimizations, are fundamentally driven by business requirements. Custom transformations may need to be developed to specifically address unique business needs and challenges. This adaptive approach ensures that the financial data engineering lifecycle effectively supports and aligns with business objectives in the dynamic financial landscape.

As you learned toward the end of this chapter, once data has been transformed and prepared for its intended purposes, it is delivered to its final consumers, seemingly marking the end of its lifecycle. At this point, you might wonder if additional steps are still required. Crucially, ensuring optimal and flawless functioning across the ingestion, storage, transformation, and delivery layers is never completely assured. This underscores the critical importance of the final layer—monitoring—which will be the focus of the next chapter.

- 1** A simple moving average is the average of the previous K data points. For example, take the vector [1,2,3,4,5]. The moving average over two data points becomes [1, 1.5, 3, 4.5, 6].
- 2** Try it yourself on [CoderPad](#).
- 3** Logarithmic speed, denoted as $O(\ln(n))$, is widely regarded as efficient in terms of performance. To illustrate this concept in simple terms, imagine you have a very

- large table and you're searching for a specific row. Now suppose that you perform your search in multiple iterations, and at each iteration, you divide the table in half and only search one half. This approach dramatically reduces the search space with each iteration. Any algorithm that operates in this manner achieves logarithmic complexity. While structures like B-trees are more complex, they share the same underlying strategy of efficiently narrowing down search areas to quickly find desired data.
- 4 For a good reference on database internals, I highly recommend Alex Petrov's [*Database Internals: A Deep-Dive into How Distributed Data Systems Work*](#) (O'Reilly, 2019).
 - 5 Some authors suggested adopting a data consumer-centric approach to data quality definition and management. To learn more about this topic, I highly recommend Richard Y. Wang and Diane M. Strong's ["Beyond Accuracy: What Data Quality Means to Data Consumers"](#), *Journal of Management Information Systems* 12, no. 4 (1996): 5–33.
 - 6 For more on this topic, check the excellent article by Lou Lindley, ["Working with High-Frequency Market Data: Data Integrity and Cleaning \(Part 1\)"](#).
 - 7 For a practical guide to this topic, I recommend reading Jason Brownlee's articles on the [de-trending](#) and [de-seasonalization](#) of time series.
 - 8 In financial data, several seasonal behaviors have been observed. For example, the [January Effect](#) refers to a seasonal pattern where prices of stocks tend to increase during the month of January of each year.
 - 9 To read more about this topic, I recommend John Y. Campbell, Andrew W. Lo, and A. Craig MacKinlay's *The Econometrics of Financial Markets*, vol. 2. (Princeton University Press, 1997).
 - 10 For more on this topic, see William M. Cready and Ramachandran Ramanan's ["The Power of Tests Employing Log-Transformed Volume in Detecting Abnormal Trading"](#), *Journal of Accounting and Economics* 14, no. 2 (June 1991): 203–214.
 - 11 Factors are often employed in an investment strategy called factor investing. To learn more about this, I recommend David Blitz and Milan Vidojevic's ["The Characteristics of Factor Investing"](#), *Journal of Portfolio Management* 45, no. 3 (2019): 69–86.
 - 12 For an insightful overview of financial markets, I recommend reading Nikita Ivanov, ["In-Memory Computing Can Digitally Transform Financial Services and FinTech Applications for Capital Markets"](#), *Forbes*, January 5, 2021.
 - 13 For more on this topic, I recommend reading A. Madhavi and T. Sivaramireddy's ["Real-Time Credit Card Fraud Detection Using Spark Framework"](#), in *Machine Learning Technologies and Applications: Proceedings of ICACECS 2020* (Springer, 2021): 287–298.
 - 14 For more context regarding this issue, I recommend the paper by John W. Lockwood, Adwait Gupte, Nishit Mehta, Michaela Blott, Tom English, and Kees Visser, ["A Low-Latency Library in FPGA Hardware for High-Frequency Trading \(HFT\)"](#), in the *2012 IEEE 20th Annual Symposium on High-Performance Interconnects* (IEEE, 2012): 9–16.

15 For more on database throughput, see Felipe Cardeneti Mendes, Piotr Sarna, Pavel Emelyanov, and Cynthia Dunlop's *Database Performance at Scale: A Practical Guide* (Apress, 2023): 254.